Dry Run

1. The Story

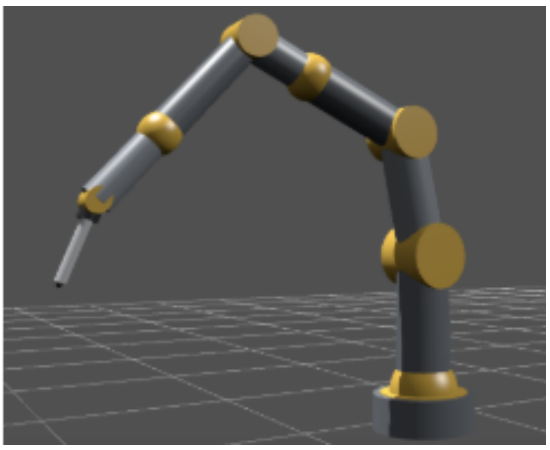
Vantage Robotics builds and sells a 6-axis industrial robotic arm used on factory floors for precision assembly and inspection work. Every arm that ships runs on the same control software, and that software is under constant development — new motion modes, new interfaces, new automation features, all being written and tested continuously.

The problem is where that testing happens. Right now, every change to the control software — a tweak to the IK solver, a new control scheme, a new automated routine — gets tested on an actual arm on the floor. That is slow: engineers queue up for limited arm time. It is risky: a bug in untested motion code can crash the arm into a fixture, damage the tooling, or injure someone standing nearby. And it is expensive: every hour the arm spends running test code is an hour it isn't doing production work.

Vantage's engineering leadership has decided this has to change. Before any control software touches a real arm, it should be built and proven entirely in a browser-based simulation — one that behaves like the real thing closely enough that engineers, operators, and even non-technical stakeholders can trust what they see in it. Only software that passes muster in simulation gets a shot on real hardware.

Your team has been brought in to build that simulation and control suite. Vantage has handed over the URDF model of their arm and the fixed coordinates of a standard test fixture — a 6-key panel used across their test rigs to validate precision and repeatability. Everything else — visualizing the arm, driving it by hand in multiple ways, commanding it by voice or natural language, and proving it can complete a precise task fully on its own — is what your team is building.

Vantage's floor supervisors are not always the engineers writing the control code. They've asked for one more thing: could an operator eventually just talk to the arm ? Like, describe what they want in plain language, have it understood correctly, and hear back a clear confirmation of what was done (or a clear, spoken heads-up when something couldn't be done). Teams that want to push further can build toward that vision as an optional, separately-scored extension of the same pipeline.



2. The Problem Statement

Build a web application that lets Vantage's engineers visualize, manually control, and eventually automate a 6-DOF industrial robotic arm — entirely in a browser, with no real hardware involved.

The arm has no gripper — only a fixed stylus tip, standard across Vantage's test rigs. A 6-key test panel sits at known, fixed coordinates relative to the arm's base. Your application must let an engineer drive the arm manually through several control methods, then extend that same motion pipeline into an autonomous mode that completes a precise task — entering a given PIN by touching the correct keys in sequence — entirely on its own.

This is, at its core, a single motion-control pipeline reused across five different ways of triggering it: a dashboard, a GUI joystick, a keyboard, a voice command, and an autonomous sequence. If this pipeline is solid and trustworthy in simulation, Vantage can be confident handing the same software a real arm. Everything you build should point back to that one pipeline.

Optional extension — Agentic Voice Control. Teams that complete the core pipeline may optionally extend it further: instead of mapping a fixed set of keyword commands to motion, route spoken (or typed) natural language through a reasoning layer — of any kind you choose — that interprets free-form, multi-step, or ambiguous instructions, converts them into the same structured motion commands your pipeline already understands, and responds back to the operator in natural language — by confirming what it understood, and reporting whether the action succeeded or, if it failed or couldn't be attempted & why.

3. What You're Given vs. What You Build

Provided by Organizers	Built by Teams
URDF file of the 6-DOF robotic arm (no gripper, fixed stylus end-effector)	Web-based 3D dashboard to visualize and move the arm
Fixed 3D coordinates of the 6-key test panel (in the arm's base frame) file is named as ***key.config.json***	A visual representation of the 6-key test panel, placed at the given coordinates and Inverse kinematics solver.
This problem statement & judging rubric	Joystick-style GUI control & Keyboard-based control
	Voice-based control
	Autonomous PIN-entry model
	Arm's electrical Schematic
	(Optional) Agentic natural-language voice to control layer

4. Tasks

Phase 1 — See the Arm

1. Load and render the provided URDF in a web-based 3D viewer (*e.g.* Three.js + urdf-loader).
2. Build a dashboard showing the arm's current joint angles and end-effector position, updating live.
3. Render the 6-key test panel in the same 3D scene, using the coordinates from key.config.json. A simple representation is enough — e.g. six colored boxes at the given positions — as long as it's placed accurately and clearly visible next to the arm.

Phase 2 — Move the Arm

1. Implement inverse kinematics: given a target (x, y, z) for the stylus tip, compute the joint angles needed to reach it.
2. Build an on-screen joystick-style control that lets the operator jog the end-effector in real time.
3. Add keyboard controls (e.g. arrow keys / WASD + modifiers) as an alternative way to jog the same motion.

Phase 3 — Talk to the Arm

1. Add voice control, mapping spoken commands ("move up", "move left", "rotate base 30 degrees") to the same motion pipeline used above. How speech is captured and understood is entirely up to you.
2. *(Optional, separately scored — see **Phase 3B** below) Extend this into a full agentic, conversational control layer.*

Phase 4 — Let the Arm Work on Its Own

1. Given a 6-digit PIN as input, make the arm autonomously sequence through the correct test-panel coordinates (using the config file provided) and perform a downward touch motion at each key, in order.

2. A key press is successful when the stylus tip reaches within a defined tolerance (e.g. ±5mm) of the target key's coordinates. Physics-based collision/contact detection is not required — this is a kinematic reach-and-touch check, not a physics simulation.

Phase 5 — Arm's Electrical Schematic

1. A 6-DOF robotic arm powered by servo motors and remotely controlled over Wi-Fi. Develop a suitable proof-of-concept (PoC) electrical circuit diagram.

5. Optional Extension

Phase 3B — Agentic Voice Control

1. Extend your fixed command-mapping with a reasoning layer of your choice (an LLM or any agentic framework) that interprets free-form, multi-step, or ambiguous spoken/typed instructions and converts them into the same structured motion commands your pipeline already executes (e.g. "nudge the tip a couple centimeters toward the panel and tap the 5 key twice").
2. The reasoning layer must respond back to the operator — in natural language, and optionally in speech — confirming what it understood, and then reporting the outcome: that the action succeeded, or, if it failed, a clear failure notice explaining why or maybe ask a clarifying question, instead of guessing, when the instruction itself is ambiguous.
3. Every command the reasoning layer produces must pass through a deterministic safety check (reachability, joint limits, workspace bounds) in your existing pipeline before any motion executes. Out-of-bounds or malformed output must be rejected or re-prompted, never executed blindly.
4. You are free to choose any speech-to-text, text-to-speech, LLM, or agent framework/provider — hosted, local, open-source, or API-based. There is no restriction on which tools you use here; judges are evaluating the resulting behavior, not your toolchain.

6. Constraints & Assumptions

- No real hardware, by design — this mirrors Vantage's actual policy: nothing gets near a physical arm until it's proven in simulation. Everything here runs in-browser using the provided URDF.
- The end-effector is a fixed stylus, matching the standard tooling on Vantage's test rigs; there is no gripper and no grasping task.
- The test panel has no visual asset provided — only its coordinates (key.config.json). Teams are responsible for rendering it; a simple representation (e.g. six colored boxes) is sufficient.
- Digit labels on the panel are not required for the core deliverable. Attempt as a bonus.
- Input to the autonomous mode is always a 6-digit PIN.
- Teams may use any web framework/library for the frontend; any language for a backend/IK service if one is used.
- Teams may use any speech recognition, speech synthesis, LLM, or agent framework/provider for the optional agentic extensions (Phase 3B) — hosted, local, open-source, or commercial. Organizers do not mandate or favor any specific tool, model, or vendor here.
- Teams pursuing the optional agentic extension are responsible for their own API keys, model access, and any associated cost.
- The optional agentic extension (Phase 3B) does not replace the required deterministic voice control (Phase 3) — baselines must still work independently and will be judged as such.
- Any command produced by an agentic/reasoning layer must pass through a deterministic safety/validation check before it is allowed to move the arm — an ungated agent that can send arbitrary or unchecked motion commands will be marked down under Architecture & Safety, regardless of how capable it otherwise appears.

7. Evaluation Criteria

Criterion	What We're Looking For	Weight
Visualization & Dashboard	URDF loads correctly; arm renders in 3D; joint states are visible and updated live.	15%
Inverse Kinematics	Given a target position, the arm computes and reaches it correctly and smoothly.	15%
Manual Control (GUI Joystick + Keyboard)	Both control modes work, feel responsive, and map intuitively to arm motion.	10%
Voice Control	**Speech commands** in natural language are correctly recognized and translated into accurate arm movement.	15%
Autonomous PIN Entry	Given a PIN, the arm sequences correctly to each key's known coordinate and "presses" it.	20%
Arm's Electrical Schematic	Correctly shows power delivery, microcontroller/driver stage, and Wi-Fi link; connections labeled and logically consistent. Show a wokwi circuit diagram	5%
Overall System Architecture & Concept Explanation	A high level system workflow with proper explanation and understanding. Need to explain rationalities to the judges	15%
Overall Polish & Presentation	UI/UX quality, code clarity, and how well the team tells the story of their build.	5%
Agentic Bonus Phase - 3B	Accuracy of NLP to motion, speech feedback quality, validation robustness.	+10% (bonus)

Core rubric totals 100%. The Agentic Extension is scored independently as bonus credit on top of the 100%, so teams who complete only the required core, are not penalized for skipping it, and teams who attempt it well can exceed 100%. Bonus features may earn extra credit at judges' discretion but are not required to score well in any category above.

8. Deliverables

- A working web application demonstrating Phases 1–5.
- Source code repository (should include diagrams and Arm’s electrical schematic)
- A deployed URL (Bonus)
- A short live demo video (Bonus), as if presenting to Vantage's engineering team: show the arm being visualized, driven manually (joystick, keyboard), driven by voice (free-form spoken - if implemented), and then given a PIN to type autonomously throughout, proving the pipeline is ready to trust with real hardware.